Logistic Regression: Diabetes Prediction

VS Code setup, execution steps, results, and viva preparation

Project files: logistic_regression.py, data/diabetes.csv, Logistic_Regression.ipynb, requirements.txt. The Python script has been tested successfully with the supplied dataset.

1. Open the project in VS Code

1. Extract the downloaded Logistic-Regression.zip file to a normal folder.

2. Open Visual Studio Code. Select File > Open Folder and choose the extracted Logistic-Regression folder.

3. Install the Microsoft Python extension if VS Code asks.

4. Open Terminal > New Terminal.

5. Create a virtual environment by running: **py -m venv .venv** (Windows). If the py command is unavailable, use **python -m venv .venv**.

6. Activate it in PowerShell: **.venv\Scripts\Activate.ps1**. In Command Prompt use **.venv\Scripts\activate.bat**.

7. Install packages: **python -m pip install -r requirements.txt**.

8. Press Ctrl+Shift+P, choose Python: Select Interpreter, and select the Python interpreter inside the .venv folder.

9. Run the project: **python logistic_regression.py**.

If PowerShell blocks activation

Switch the VS Code terminal profile to Command Prompt and run .venv\Scripts\activate.bat. You can also bypass activation and run .venv\Scripts\python.exe logistic_regression.py after installing packages with .venv\Scripts\python.exe -m pip install -r requirements.txt.

2. What the program does

• Loads 768 records from data/diabetes.csv. The target column is Outcome: 0 means the record is labelled no diabetes and 1 means diabetes.

• Uses six input features: Age, BMI, BloodPressure, Insulin, Glucose, and DiabetesPedigreeFunction.

• Splits data into 80% training and 20% testing data. random_state=42 makes the split repeatable; stratify=y preserves the target class proportions.

• StandardScaler standardizes feature values so variables measured on different scales do not dominate model fitting.

• LogisticRegression learns a binary classification model. predict() returns the predicted class, while predict_proba() returns class probabilities.

• Evaluates predictions with accuracy, a classification report, a confusion matrix, and ROC-AUC. It saves two charts and a coefficient CSV in outputs/.

3. Verified run from this dataset

Dataset: 768 rows, 8 columns. **Training:** 614 rows. **Testing:** 154 rows.

Accuracy: 0.7338 (73.38%). **ROC-AUC:** 0.8093.

Confusion matrix: [[84, 16], [25, 29]] using class order [No Diabetes, Diabetes]. This means 84 true negatives, 16 false positives, 25 false negatives, and 29 true positives.

Your output should match when the same versions and files are used. Small differences can occur with different package versions.

4. Output files

• outputs/confusion_matrix.png - visual comparison of actual and predicted classes.

• outputs/roc_curve.png - ROC curve with AUC score.

• outputs/model_coefficients.csv - learned coefficients in standardized feature space.

5. Viva questions and short answers

Q: What is Logistic Regression?
A: A supervised machine-learning algorithm used mainly for classification. It estimates the probability that an input belongs to a class.

Q: Why is it called regression if it is used for classification?
A: It models a continuous probability using a regression equation, then applies a threshold to classify the result.

Q: What is the sigmoid function?
A: It converts any real-valued score into a value between 0 and 1: sigmoid(z) = 1 / (1 + e^(-z)).

Q: What is the target variable here?
A: Outcome. Zero represents the no-diabetes class and one represents the diabetes class in this dataset.

Q: What are features?
A: Input columns used by the model to make a prediction. Here there are six selected health-related features.

Q: Why split the dataset?
A: Training data teaches the model; test data checks performance on records not used during training.

Q: Why use 80/20?
A: It is a common practical split that leaves most records for learning and a separate portion for evaluation.

Q: What does random_state=42 do?
A: It makes the random train/test split reproducible.

Q: Why use StandardScaler?
A: It puts numeric features on a comparable scale, which helps optimization and makes fitting more stable.

Q: What is max_iter=1000?
A: It allows the Logistic Regression optimizer up to 1000 iterations to converge.

Q: What is accuracy?
A: The fraction of all test predictions that are correct.

Q: What is a confusion matrix?
A: A table comparing actual and predicted classes. For binary classification it shows TN, FP, FN, and TP.

Q: What is a false negative?
A: The actual class is positive but the model predicts negative. Here it means a diabetes-labelled record predicted as no diabetes.

Q: What is precision?
A: Among records predicted positive, the fraction that are truly positive.

Q: What is recall?
A: Among all actual positive records, the fraction correctly predicted positive.

Q: What is ROC-AUC?
A: A measure of how well the model separates the two classes across classification thresholds. 1 is ideal; 0.5 is roughly random ranking.

Q: What does predict_proba() return?
A: Estimated probabilities for each class. The script uses the probability of class 1 to calculate ROC-AUC.

Q: Is this model suitable for medical diagnosis?
A: No. This is an educational demonstration on a dataset, not a clinically validated diagnostic system.

6. Important explanation of the result

The accuracy of 73.38% means 113 of 154 test records were classified correctly (84 + 29). The model missed 25 positive records in this test split, so accuracy alone does not tell the whole story. In health-related classification, recall and false negatives matter too. Explain the limitations rather than claiming the model is perfect.

7. Troubleshooting

• ModuleNotFoundError: run python -m pip install -r requirements.txt in the selected .venv terminal.

• FileNotFoundError: confirm data/diabetes.csv exists inside the project folder, not beside the folder.

• Wrong Python interpreter: use Ctrl+Shift+P > Python: Select Interpreter and choose .venv.

• Notebook kernel missing: install packages from requirements.txt, then select the .venv kernel in the notebook's kernel picker.

Viva tip: explain the problem, dataset, features, train/test split, scaling, model training, evaluation metrics, and one limitation in that order. Keep the terminal output open during your demonstration.